MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD

Prayagraj, Uttar Pradesh, India

Department of Electronics and Communication Engineering

PROJECT REPORT

on

SPI EEPROM MOTOR-PROFILE CONTROLLER

Storage, Retrieval and Execution of DC Motor Speed Profiles using an LPC2138 Microcontroller, a 25LC256 SPI EEPROM and an L293D Motor Driver

Project	SPI EEPROM Motor-Profile Controller
Microcontroller	NXP LPC2138 (ARM7TDMI-S)
Profile memory	Microchip 25LC256, 256 Kbit SPI EEPROM
Motor stage	L293D H-bridge driver and DC motor
Firmware / tools	Embedded C and ARM assembly, Arm GNU Toolchain
Simulation	Proteus Design Suite

SUBMITTED BY

Team 1

S. No.	Name of Student	Roll Number
1	Anurag Nirmal	20244031
2	Arin Saraf	20244034
3	Arun Kumar	20244038
4	Ashish Kumar Kashyap	20244043
5	Ashish Pandey	20244044

SUBMITTED TO

Dr. Shishir Maheshwari

Department of Electronics and Communication Engineering

Motilal Nehru National Institute of Technology Allahabad

Academic Session: 2026–2027

ABSTRACT

This report presents an **SPI EEPROM Motor-Profile Controller** built around the NXP **LPC2138** ARM7 microcontroller. A motor profile — a list of steps, each with a direction, speed, ramp time and run time — is stored in a Microchip **25LC256** SPI EEPROM, read back and validated at power-up, and executed on a DC motor through an **L293D** H-bridge driver.

The firmware is written in bare-metal C with an ARM assembly start-up file and is built with the Arm GNU Toolchain. SPI0 runs as a master in mode 0,0; the EEPROM driver uses write-enable checks, write-cycle polling and time-outs, and the profile is protected by a version number and checksum. Motor speed is set by a 100 Hz PWM signal with speed ramps and a pause before every change of direction. In Proteus, an EEPROM write/read self-test passed, and the final firmware ran the motor with the simulation keeping pace with real time. Testing on physical hardware remains future work.

Keywords: LPC2138, ARM7TDMI-S, SPI, 25LC256 EEPROM, L293D, PWM, DC motor control, non-volatile parameter storage, Proteus simulation, bare-metal firmware.

ACKNOWLEDGEMENT

We express our sincere gratitude to **Dr. Shishir Maheshwari**, Department of Electronics and Communication Engineering, Motilal Nehru National Institute of Technology Allahabad, for his guidance and for giving us the opportunity to work on this project.

We also thank the Department of Electronics and Communication Engineering, MNNIT Allahabad, for providing the academic environment and resources that made this work possible. Finally, we thank our classmates and families for their encouragement throughout the project.

Team 1
Department of ECE, MNNIT Allahabad

TABLE OF CONTENTS

ABSTRACT ..... **i**
ACKNOWLEDGEMENT ..... **i**
1. INTRODUCTION **1**
1.1 Project Overview 1
1.2 Motivation and Scope 1
1.3 Development Environment 1
2. OBJECTIVES **1**
3. TECHNICAL BACKGROUND **1**
3.1 LPC2138 Microcontroller 1
3.2 SPI Interface 2
3.3 25LC256 Serial EEPROM 2
3.4 PWM Motor Control with the L293D 2
4. SYSTEM ARCHITECTURE AND HARDWARE DESIGN **2**
4.1 Block Diagram 2
4.2 Components and Pin Assignment 3
4.3 Power Supply 4
4.4 Proteus Schematic 4
5. EEPROM PROFILE FORMAT **4**
6. WORKING PRINCIPLE AND DESIGN CALCULATIONS **5**
6.1 Operating Sequence 5
6.2 Design Calculations 5
7. SOFTWARE DESIGN **6**
7.1 Build Process 6
7.2 Running the Simulation 6
7.3 Firmware Structure 7
7.4 Error Handling 7
8. SOURCE CODE **8**
8.1 Motor-Profile Controller (main.c) 8
8.2 Motor Driver Test (motor_test.c) 12
9. RESULTS AND DISCUSSION **13**
9.1 EEPROM Communication 13
9.2 Motor-Profile Execution 13
9.3 Problems Faced and Solutions 14
9.4 Discussion 15
10. APPLICATIONS AND FUTURE IMPROVEMENTS **15**
10.1 Applications 15
10.2 Limitations and Future Improvements 15
11. CONCLUSION **15**
REFERENCES **16**

LIST OF FIGURES

Figure 4.1: System block diagram 3
Figure 4.2: Proteus schematic of the complete system during simulation 4
Figure 5.1: EEPROM memory map of the stored profile 4
Figure 6.1: Software PWM: Timer0 count and the EN1 signal at 75 % and 40 % duty 6
Figure 7.1: Firmware flowchart 7
Figure 9.1: L293D pin states and the rotating motor (detail of Figure 4.2) 13
Figure 9.2: Proteus status bar during the same run 13
Figure 9.3: Motor drive over one profile cycle, measured in emulation 14
Figure 9.4: Reverse step of the profile in Proteus (to be captured) 14
Figure 9.5: Earlier run with 3 kHz PWM: IN1 high, motor still at 0.00 15

LIST OF TABLES

Table 1.1: Development environment and tools 1
Table 3.1: SPI0 registers and values used 2
Table 3.2: 25LC256 instructions used 2
Table 3.3: L293D control as used in this design 2
Table 4.1: Components 3
Table 4.2: Pin assignment 3
Table 4.3: Supply connections 4
Table 5.1: Profile byte layout 5
Table 5.2: Default motor profile 5
Table 6.1: Design calculations 5
Table 7.1: Main firmware functions 7
Table 7.2: Status indication 8
Table 9.1: Main problems and how they were solved 14

1. INTRODUCTION

1.1 Project Overview

Many motor-driven systems repeat the same sequence of speeds and directions. If that sequence is fixed in the program, every change means rebuilding and reprogramming the firmware. In this project the sequence, called a **motor profile**, is stored in an external SPI EEPROM. At power-up the LPC2138 microcontroller reads the profile, checks that it is valid, and runs a DC motor through an L293D driver accordingly.

All work was carried out in bare-metal firmware and in the Proteus Design Suite simulator on a Windows PC. Physical hardware was not built.

1.2 Motivation and Scope

- **Non-volatile settings:** motor parameters survive power loss without reprogramming the microcontroller.
- **Register-level SPI:** configuring SPI0 as a master and writing a complete EEPROM driver.
- **Safe and robust control:** speed ramps, a pause before every direction change, time-outs on every wait and clear error reporting.

Closed-loop speed control, a user interface for editing profiles and physical hardware are outside the present scope.

1.3 Development Environment

Table 1.1: *Development environment and tools*

Item	Details
Host system	Windows PC; files edited in Notepad; builds run from the Command Prompt
Toolchain	Arm GNU Toolchain 15.2.Rel1: `arm-none-eabi-gcc`, `-as`, `-ld`, `-objcopy`
Simulator	Proteus Design Suite (LPC2138, 25LC256, L293D and MOTOR-DC models)

2. OBJECTIVES

1. Configure LPC2138 SPI0 as a master and establish two-way communication with the 25LC256 EEPROM.
2. Implement an EEPROM driver with write enable, status polling, page-aware writes and time-outs.
3. Define a validated storage format for a multi-step motor profile.
4. Load the profile at power-up, writing and verifying a default profile when none is valid.
5. Drive a DC motor through an L293D with PWM speed control, ramps and safe direction reversal.
6. Indicate status clearly with an LED and debug variables, and verify the design in Proteus.

3. TECHNICAL BACKGROUND

3.1 LPC2138 Microcontroller

The NXP LPC2138 is a 32-bit ARM7TDMI-S microcontroller with 512 KB flash, 32 KB RAM and a maximum clock of 60 MHz. Its peripherals run from the peripheral clock PCLK, which equals CCLK/4 after reset (`VPBDIV` = 0). Each pin can take one of up to four functions, selected by two bits in `PINSEL0`/`PINSEL1`. This project uses GPIO port 0, SPI0, Timer0 and Timer1.

3.2 SPI Interface

SPI is a synchronous, full-duplex bus: the master drives the clock SCK and data line MOSI, receives on MISO, and selects the slave with an active-low chip select. In **mode 0,0** the clock idles low and data are sampled on the rising edge. Every byte sent also clocks one byte in, so reading requires sending a dummy byte. On the LPC2138, writing `S0SPDR` starts a transfer and the SPIF flag in `S0SPSR` marks its end. Here the chip select is an ordinary GPIO pin (P0.7) driven by the firmware.

Table 3.1: *SPI0 registers and values used*

Register	Address	Value used
S0SPCR	0xE002 0000	0x20: master, mode 0,0, MSB first
S0SPSR	0xE002 0004	Polled for SPIF (bit 7); MODF/ABRT treated as errors
S0SPDR	0xE002 0008	Data register
S0SPCCR	0xE002 000C	8: SCK = PCLK / 8

3.3 25LC256 Serial EEPROM

The Microchip 25LC256 stores 32 KB, runs from 2.5 V to 5.5 V and is organised in 64-byte pages. A write needs a WREN instruction first, which sets the write-enable latch (WEL), and takes up to 5 ms, during which the WIP status bit is set. Writes must not cross a page boundary. The instructions used are listed in Table 3.2.

Table 3.2: *25LC256 instructions used*

Instruction	Opcode	Purpose
WREN	0x06	Enable writing (sets WEL)
RDSR	0x05	Read status: WIP (bit 0), WEL (bit 1), block protection (bits 2–3)
WRITE	0x02	Write 1–64 bytes within one page
READ	0x03	Read any number of bytes from an address

3.4 PWM Motor Control with the L293D

With pulse-width modulation, a motor supply V_S is switched on for a fraction D of each period, so the motor sees an average voltage of about D × V_S. The L293D H-bridge sets the direction with IN1/IN2, and its enable input EN1 receives the PWM signal. Its logic is supplied at 5 V (VSS) and its motor bridge at 12 V (VS).

Table 3.3: *L293D control as used in this design*

EN1	IN1	IN2	Motor
PWM	1	0	Forward
PWM	0	1	Reverse
0	0	0	Stopped (coasting)

4. SYSTEM ARCHITECTURE AND HARDWARE DESIGN

4.1 Block Diagram

The LPC2138 talks to the EEPROM over SPI0, drives a status LED, and controls the L293D with one PWM signal and two direction signals (Figure 4.1). The microcontroller and EEPROM run at 3.3 V; the L293D uses 5 V for logic and 12 V for the motor, with all grounds common.

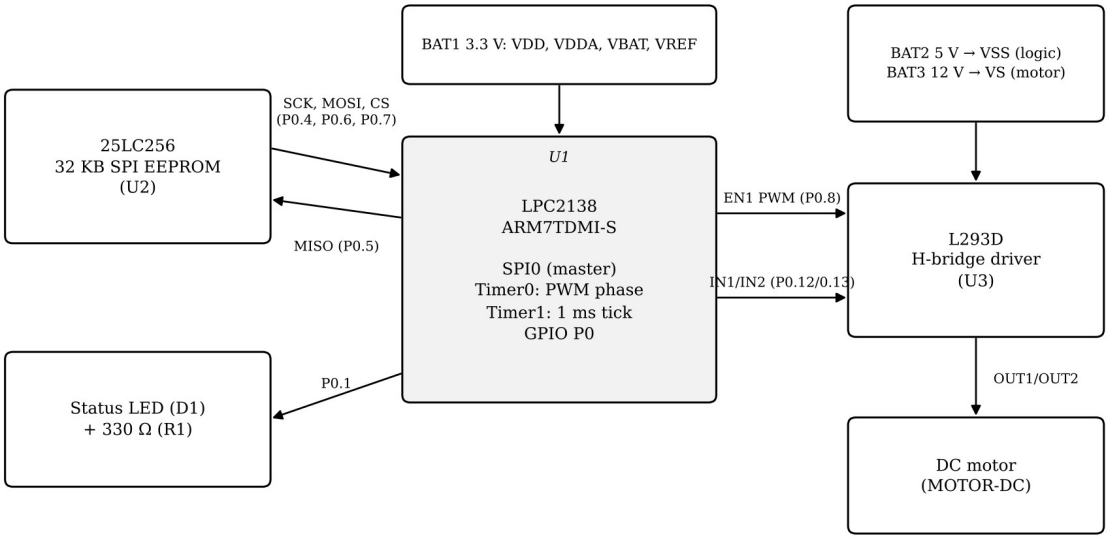


Figure 4.1: *System block diagram*

4.2 Components and Pin Assignment

Table 4.1: *Components*

Ref.	Component	Function	Supply
U1	LPC2138	Controller, SPI master, timers, GPIO	3.3 V (BAT1)
U2	25LC256	Stores the motor profile	3.3 V
U3	L293D	H-bridge motor driver	5 V logic (BAT2), 12 V motor (BAT3)
M1	MOTOR-DC	Motor (shows speed in simulation)	L293D OUT1/OUT2
D1, R1	LED, 330 Ω	Status indicator	P0.1

Table 4.2: *Pin assignment*

LPC2138 pin (no.)	Function	Connected to
P0.1 (21)	GPIO output	R1 → LED D1 → GND
P0.4 / SCK0 (27)	SPI0 clock	25LC256 SCK (pin 6)
P0.5 / MISO0 (29)	SPI0 data in	25LC256 SO (pin 2)
P0.6 / MOSI0 (30)	SPI0 data out	25LC256 SI (pin 5)
P0.7 (31)	GPIO, chip select	25LC256 CS (pin 1)
P0.8 (33)	GPIO, PWM output	L293D EN1 (pin 1)
P0.12 (38)	GPIO	L293D IN1 (pin 2)
P0.13 (39)	GPIO	L293D IN2 (pin 7)
3.3 V	—	LPC2138 VBAT, VREF, V3A, V3; EEPROM WP, HOLD, VCC

`PINSEL0` = 0x0000 1500 selects SCK0, MISO0 and MOSI0 on P0.4–P0.6 and leaves every other pin used here as GPIO.

4.3 Power Supply

Table 4.3: *Supply connections*

Supply	Connected to
BAT1, 3.3 V	LPC2138 VBAT (49), VREF (63), V3A (7), V3 (51, 43, 23); EEPROM VCC, WP, HOLD
BAT2, 5 V	L293D VSS (pin 16), logic supply
BAT3, 12 V	L293D VS (pin 8), motor supply
Ground	All battery negatives, LPC2138 VSS/VSSA, EEPROM VSS, L293D GND and EN2, LED cathode

Every LPC2138 supply pin, including VBAT, must be at 3.3 V; leaving VBAT unconnected or putting VDD on a 5 V net stops the simulation. The L293D logic needs at least 4.5 V, so it has its own 5 V supply, and it accepts the LPC2138's 3.3 V signals on its inputs.

4.4 Proteus Schematic

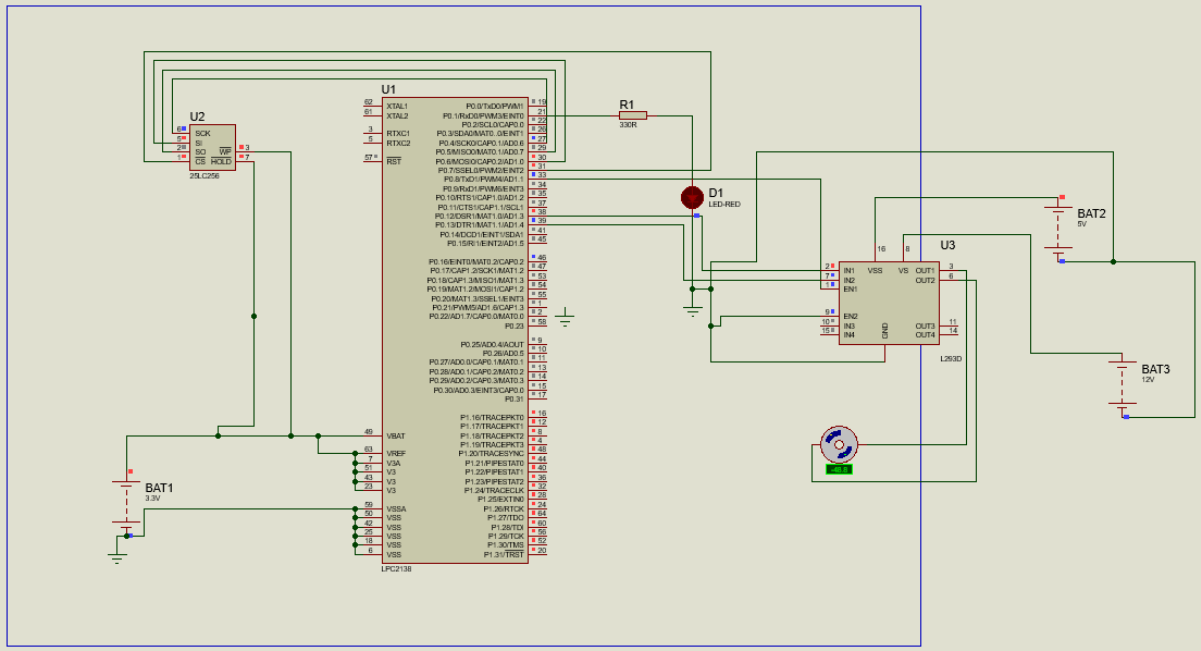


Figure 4.2: *Proteus schematic of the complete system during simulation*

5. EEPROM PROFILE FORMAT

A profile is a list of up to eight steps. Each step gives a direction (stop, forward or reverse), a speed in percent, a ramp time and a run time. The profile is stored from address 0x0000 as shown in Figure 5.1 and Table 5.1; the default four-step profile takes 29 bytes and is written with a single page write.

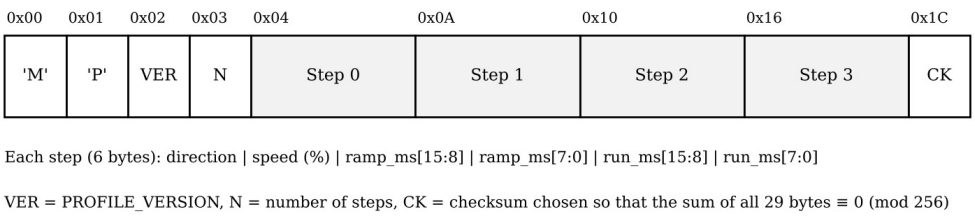


Figure 5.1: *EEPROM memory map of the stored profile*

Table 5.1: *Profile byte layout*

Offset	Field	Rule
0x00–0x01	Magic	'M', 'P'
0x02	Version	Must equal `PROFILE_VERSION` (2)
0x03	Step count N	1 to 8
0x04 + 6k	Step k	direction, speed %, ramp_ms (2 bytes), run_ms (2 bytes)
0x04 + 6N	Checksum	Makes the sum of all bytes 0 (mod 256)

Table 5.2: *Default motor profile*

Step	Direction	Speed	Ramp	Run
0	Forward	75 %	500 ms	3000 ms
1	Stop	0 %	300 ms	1000 ms
2	Reverse	40 %	500 ms	3000 ms
3	Stop	0 %	300 ms	1000 ms

At power-up the profile is accepted only if the magic bytes, version, step count, value ranges and checksum are all correct. Otherwise the default profile is written and read back, so the motor always runs data that came from the EEPROM. Increasing `PROFILE_VERSION` forces new defaults to be stored.

6. WORKING PRINCIPLE AND DESIGN CALCULATIONS

6.1 Operating Sequence

1. Start-up code sets the stack, prepares RAM and calls `main()`.
2. Pins, Timer1 (1 ms) and Timer0 (PWM) are set up; the LED blinks three times.
3. SPI0 is started and the EEPROM status is checked.
4. The profile is read and validated, or the default is written and read back.
5. On success the LED stays on and the steps run in a loop; on any error the motor stays off and the LED blinks at 5 Hz.

6.2 Design Calculations

No PLL is used and the U1 clock in Proteus is 12 MHz, so PCLK = 12 MHz / 4 = **3 MHz**. The main values follow from this:

Table 6.1: *Design calculations*

Quantity	Calculation	Result
SPI clock	PCLK / S0SPCCR = 3 MHz / 8	375 kHz
SPI byte time	8 / 375 kHz	≈ 21.3 µs
Timer1 tick	T1PR = 3 MHz / 1000 − 1 = 2999	1 ms
PWM step	T0PR = 3 MHz / 10 000 − 1 = 299	100 µs
PWM period / frequency	100 steps × 100 µs (T0MR0 = 99)	10 ms / 100 Hz
PWM resolution	1 / 100	1 %
Motor voltage, forward	0.75 × 12 V (ideal)	9.0 V
Motor voltage, reverse	0.40 × 12 V (ideal)	4.8 V
Forward ramp	⌊500 / 75⌋ = 6 ms per 1 % × 75	450 ms

Quantity	Calculation	Result
One profile cycle	3450 + 1500 + 3480 + 1480 ms	≈ 9.9 s

The firmware generates PWM in software: while waiting, it compares the Timer0 count (0–99) with the duty value and drives EN1 high while the count is lower (Figure 6.1). An earlier version used a 3 kHz hardware PWM, which made the Proteus motor simulation run far slower than real time. At 100 Hz there are about 30 times fewer switching edges to simulate.

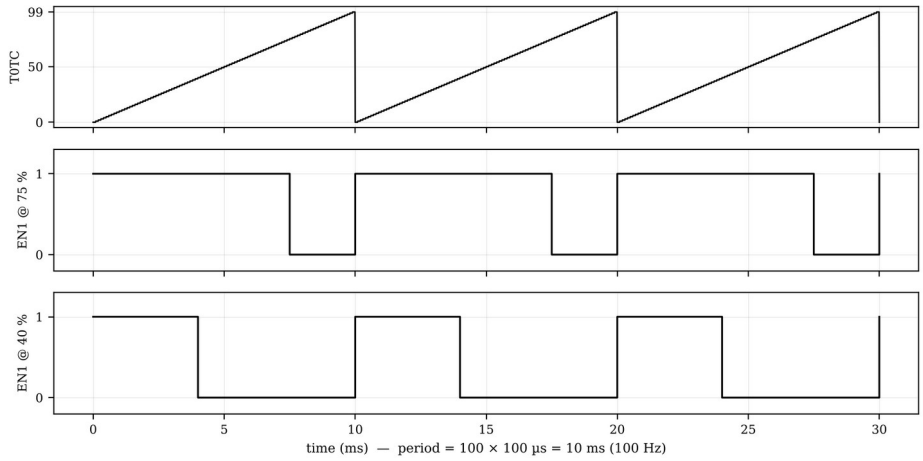


Figure 6.1: *Software PWM: Timer0 count and the EN1 signal at 75 % and 40 % duty*

7. SOFTWARE DESIGN

7.1 Build Process

The firmware is built from the Command Prompt with the following commands. The ELF file can be loaded into Proteus directly, so that variables can be watched by name.

```
arm-none-eabi-as  -mcpu=arm7tdmi -g -o startup.o startup.s
arm-none-eabi-gcc -c -mcpu=arm7tdmi -marm -O0 -g -Wall -Wextra \
                  -ffreestanding -fno-builtin main.c -o main.o
arm-none-eabi-ld  -T linker.ld -o firmware.elf startup.o main.o
arm-none-eabi-objcopy -O ihex firmware.elf firmware.hex
```

The linker script places code in flash at 0x0000 0000 and data in RAM at 0x4000 0000, with the stack at the top of RAM. Because no C library is linked and the ARM7 has no divide instruction, the firmware includes its own division routine, `udiv()`.

7.2 Running the Simulation

1. Build the firmware; the build produces `firmware.elf` and `firmware.hex`.
2. In Proteus, open the properties of U1, set Program File to `firmware.elf` and Clock Frequency to 12 MHz.
3. Set the motor's nominal voltage to 12 V and check the supply voltages of BAT1–BAT3.
4. Start the simulation and watch the simulated time in the status bar rather than the wall clock.
5. To see the debug variables, pause and open Debug → Watch Window, then add `g_fail_code`, `g_step`, `g_direction` and `g_speed`.

7.3 Firmware Structure

Table 7.1: *Main firmware functions*

Functions	Purpose
`spi_transfer`	Send and receive one SPI byte, with a time-out
`ee_read_status`, `ee_wait_ready`, `ee_check`	EEPROM status, write-complete polling, presence check
`ee_write_page`, `ee_write_block`, `ee_read_block`	Write enable and page writes; block reads
`profile_encode`, `profile_decode`, `profile_load`	Build, check and load the stored profile
`delay_ms`, `pwm_service`	1 ms delays that keep the software PWM running
`motor_set_speed`, `motor_set_direction`, `motor_ramp`, `motor_run_step`	Speed, direction, ramps, one profile step

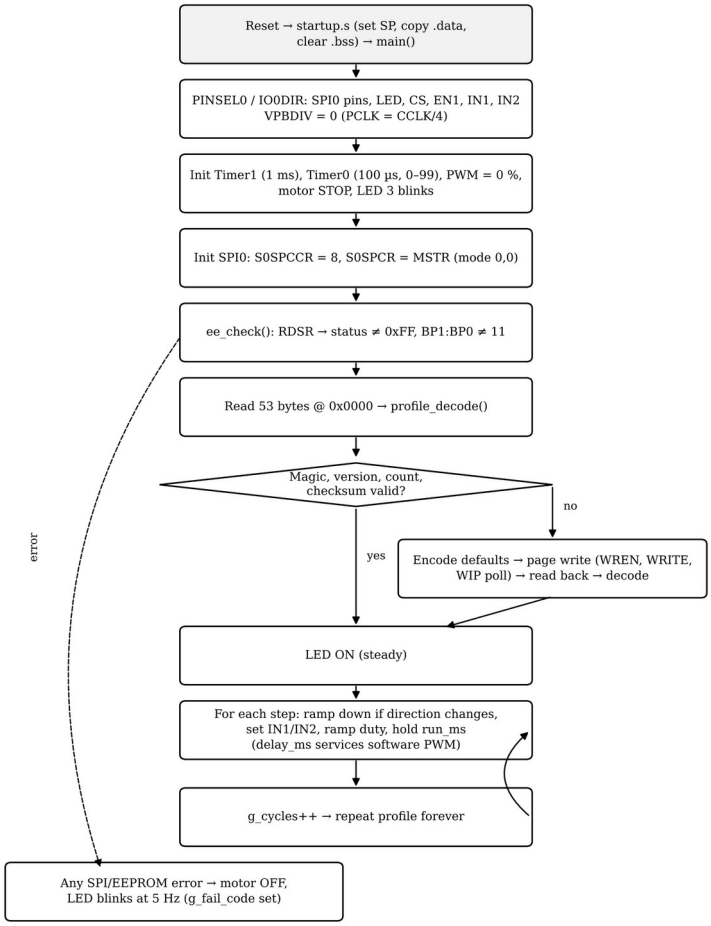


Figure 7.1: *Firmware flowchart*

7.4 Error Handling

Every SPI wait and every EEPROM write-cycle wait has a time-out. Before each write the firmware confirms that the WEL bit is set, which also proves that the EEPROM is answering on MISO. Errors stop the motor and are reported through the LED and debug variables (Table 7.2). A change of direction is always preceded by a ramp to zero and a 200 ms pause.

Table 7.2: *Status indication*

Indication	Meaning
3 blinks, then steady LED	Firmware started; profile loaded; motor running
LED blinking at 5 Hz	EEPROM error, motor off; cause in `g_fail_code`
`g_fail_code` 1 / 2 / 3	SPI time-out / SPI fault / EEPROM not answering
`g_fail_code` 4 / 5 / 6 / 9	Write-protected / WEL not set / write time-out / bad profile
`g_profile_source`	1 = profile found in EEPROM, 2 = defaults written
`g_step`, `g_direction`, `g_speed`, `g_cycles`	Current step, direction, duty %, cycles completed

8. SOURCE CODE

The listings below are the final motor-profile controller (`main.c`) and a test program (`motor_test.c`) that drives the L293D directly, to separate circuit faults from firmware faults. The `main.c` listed is a shortened version of the firmware used in the Proteus runs, with the same behaviour.

8.1 Motor-Profile Controller (main.c)

Listing 8.1: main.c *(376 lines)*

```
#define USE_HW_PWM 0

#define REG(a) (*(volatile unsigned int *)(a))

#define PINSEL0 REG(0xE002C000)
#define IO0SET  REG(0xE0028004)
#define IO0DIR  REG(0xE0028008)
#define IO0CLR  REG(0xE002800C)
#define S0SPCR  REG(0xE0020000)
#define S0SPSR  REG(0xE0020004)
#define S0SPDR  REG(0xE0020008)
#define S0SPCCR REG(0xE002000C)
#define T0TCR   REG(0xE0004004)
#define T0TC    REG(0xE0004008)
#define T0PR    REG(0xE000400C)
#define T0MCR   REG(0xE0004014)
#define T0MR0   REG(0xE0004018)
#define T1TCR   REG(0xE0008004)
#define T1TC    REG(0xE0008008)
#define T1PR    REG(0xE000800C)
#define T1MCR   REG(0xE0008014)
#define PWMTCR  REG(0xE0014004)
#define PWMPR   REG(0xE001400C)
#define PWMMCR  REG(0xE0014014)
#define PWMMR0  REG(0xE0014018)
#define PWMMR4  REG(0xE0014040)
#define PWMPCR  REG(0xE001404C)
#define PWMLER  REG(0xE0014050)
#define PCONP   REG(0xE01FC0C4)
#define VPBDIV  REG(0xE01FC100)

#define CCLK_HZ 12000000U
#define PCLK_HZ (CCLK_HZ / 4U)

#define LED_PIN (1U << 1)
#define CS_PIN  (1U << 7)
#define EN1_PIN (1U << 8)
#define IN1_PIN (1U << 12)
#define IN2_PIN (1U << 13)

#define PROFILE_VERSION     2U
#define MAX_STEPS           8U
#define PROFILE_MAX_BYTES   (4U + MAX_STEPS * 6U + 1U)
#define DIR_CHANGE_PAUSE_MS 200U

enum { FAIL_NONE, FAIL_SPI_TIMEOUT, FAIL_SPI_FAULT, FAIL_NO_RESPONSE, FAIL_PROTECTED,
       FAIL_WEL_NOT_SET, FAIL_WRITE_TIMEOUT, FAIL_PROFILE_BAD = 9 };
enum { DIR_STOP, DIR_FORWARD, DIR_REVERSE };

typedef struct {
    unsigned char direction, speed;
    unsigned short ramp_ms, run_ms;
} motor_step_t;
```

```
54
55  static const motor_step_t default_steps[] = {
56      { DIR_FORWARD, 75, 500, 3000 },
57      { DIR_STOP,     0, 300, 1000 },
58      { DIR_REVERSE, 40, 500, 3000 },
59      { DIR_STOP,     0, 300, 1000 },
60  };
61  #define DEFAULT_STEP_COUNT (sizeof(default_steps) / sizeof(default_steps[0]))
62
63  static motor_step_t steps[MAX_STEPS];
64  static unsigned int step_count, pwm_duty;
65
66  volatile unsigned int g_fail_code, g_profile_source, g_step_count;
67  volatile unsigned int g_step, g_direction, g_speed, g_cycles;
68
69  static unsigned int udiv(unsigned int n, unsigned int d)
70  {
71      unsigned int q = 0, bit = 1;
72
73      if (d == 0) return 0;
74      while (d < n && !(d & 0x80000000U)) { d <<= 1; bit <<= 1; }
75      for (; bit; d >>= 1, bit >>= 1)
76          if (n >= d) { n -= d; q |= bit; }
77      return q;
78  }
79
80  static void short_delay(void)
81  {
82      volatile unsigned int i;
83      for (i = 0; i < 10; i++) { }
84  }
85
86  static void cs_low(void)  { IO0CLR = CS_PIN; short_delay(); }
87  static void cs_high(void) { short_delay(); IO0SET = CS_PIN; short_delay(); }
88
89  static void timers_init(void)
90  {
91      PCONP |= (1U << 1) | (1U << 2);
92      T1TCR = 2; T1PR = PCLK_HZ / 1000 - 1; T1MCR = 0; T1TCR = 1;
93      T0TCR = 2; T0PR = PCLK_HZ / 10000 - 1; T0MR0 = 99; T0MCR = 2; T0TCR = 1;
94  }
95
96  static void pwm_service(void)
97  {
98  #if !USE_HW_PWM
99      static unsigned int en1_is_high;
100     unsigned int want_high = pwm_duty >= 100 || (pwm_duty && T0TC < pwm_duty);
101
102     if (want_high != en1_is_high) {
103         if (want_high) IO0SET = EN1_PIN; else IO0CLR = EN1_PIN;
104         en1_is_high = want_high;
105     }
106 #endif
107 }
108
109 static void delay_ms(unsigned int ms)
110 {
111     unsigned int start = T1TC;
112     while (T1TC - start < ms) pwm_service();
113 }
114
115 static unsigned int spi_transfer(unsigned char tx, unsigned char *rx)
116 {
117     unsigned int timeout = 100000, status;
118
119     S0SPDR = tx;
120     for (;;) {
121         status = S0SPSR;
122         if (status & 0x80) break;
123         if (status & 0x18) return FAIL_SPI_FAULT;
124         if (--timeout == 0) return FAIL_SPI_TIMEOUT;
125     }
126     status = S0SPDR;
127     if (rx) *rx = (unsigned char)status;
128     return FAIL_NONE;
129 }
130
131 static unsigned int ee_read_status(unsigned char *status)
132 {
133     unsigned int err;
134
135     cs_low();
136     err = spi_transfer(0x05, 0);
137     if (!err) err = spi_transfer(0x00, status);
138     cs_high();
139     return err;
140 }
141
142 static unsigned int ee_wait_ready(void)
```

```
{
    unsigned int polls, err;
    unsigned char status;

    for (polls = 0; polls < 20000; polls++) {
        if ((err = ee_read_status(&status))) return err;
        if (!(status & 0x01)) return FAIL_NONE;
    }
    return FAIL_WRITE_TIMEOUT;
}

static unsigned int ee_send_address(unsigned char cmd, unsigned int addr)
{
    unsigned int err = spi_transfer(cmd, 0);

    if (!err) err = spi_transfer((addr >> 8) & 0x7F, 0);
    if (!err) err = spi_transfer(addr & 0xFF, 0);
    return err;
}

static unsigned int ee_write_page(unsigned int addr, const unsigned char *buf,
                                  unsigned int len)
{
    unsigned int err, i;
    unsigned char status = 0;

    cs_low();
    err = spi_transfer(0x06, 0);
    cs_high();
    if (err || (err = ee_read_status(&status))) return err;
    if (!(status & 0x02)) return FAIL_WEL_NOT_SET;

    cs_low();
    err = ee_send_address(0x02, addr);
    for (i = 0; i < len && !err; i++) err = spi_transfer(buf[i], 0);
    cs_high();
    return err ? err : ee_wait_ready();
}

static unsigned int ee_write_block(unsigned int addr, const unsigned char *buf,
                                   unsigned int len)
{
    unsigned int err, chunk;

    for (; len; addr += chunk, buf += chunk, len -= chunk) {
        chunk = 64 - (addr & 63);
        if (chunk > len) chunk = len;
        if ((err = ee_write_page(addr, buf, chunk))) return err;
    }
    return FAIL_NONE;
}

static unsigned int ee_read_block(unsigned int addr, unsigned char *buf, unsigned int len)
{
    unsigned int err, i;

    cs_low();
    err = ee_send_address(0x03, addr);
    for (i = 0; i < len && !err; i++) err = spi_transfer(0x00, &buf[i]);
    cs_high();
    return err;
}

static unsigned int ee_check(void)
{
    unsigned char status = 0;
    unsigned int err = ee_read_status(&status);

    if (err) return err;
    if (status == 0xFF) return FAIL_NO_RESPONSE;
    return (status & 0x0C) == 0x0C ? FAIL_PROTECTED : FAIL_NONE;
}

static unsigned char checksum(const unsigned char *buf, unsigned int len)
{
    unsigned char sum = 0;
    while (len--) sum += *buf++;
    return (unsigned char)(0 - sum);
}

static unsigned int profile_encode(const motor_step_t *src, unsigned int count,
                                   unsigned char *buf)
{
    unsigned int i, n = 4;

    buf[0] = 'M'; buf[1] = 'P'; buf[2] = PROFILE_VERSION; buf[3] = (unsigned char)count;
    for (i = 0; i < count; i++) {
        buf[n++] = src[i].direction;
        buf[n++] = src[i].speed;
```

```
        buf[n++] = src[i].ramp_ms >> 8; buf[n++] = src[i].ramp_ms & 0xFF;
        buf[n++] = src[i].run_ms >> 8;  buf[n++] = src[i].run_ms & 0xFF;
    }
    buf[n] = checksum(buf, n);
    return n + 1;
}

static unsigned int profile_decode(const unsigned char *buf)
{
    unsigned int i, count = buf[3];

    if (buf[0] != 'M' || buf[1] != 'P' || buf[2] != PROFILE_VERSION)
        return 0;
    if (count == 0 || count > MAX_STEPS)
        return 0;
    if (checksum(buf, 5 + count * 6) != 0) return 0;

    for (i = 0; i < count; i++) {
        const unsigned char *p = buf + 4 + i * 6;
        if (p[0] > DIR_REVERSE || p[1] > 100) return 0;
        steps[i].direction = p[0];
        steps[i].speed = p[1];
        steps[i].ramp_ms = (unsigned short)(p[2] << 8 | p[3]);
        steps[i].run_ms = (unsigned short)(p[4] << 8 | p[5]);
    }
    step_count = count;
    return 1;
}

static unsigned int profile_load(void)
{
    unsigned char buf[PROFILE_MAX_BYTES];
    unsigned int err;

    if ((err = ee_check())) return err;
    if ((err = ee_read_block(0, buf, PROFILE_MAX_BYTES))) return err;
    if (profile_decode(buf)) { g_profile_source = 1; return FAIL_NONE; }

    err = ee_write_block(0, buf, profile_encode(default_steps, DEFAULT_STEP_COUNT, buf));
    if (err || (err = ee_read_block(0, buf, PROFILE_MAX_BYTES))) return err;
    if (!profile_decode(buf)) return FAIL_PROFILE_BAD;
    g_profile_source = 2;
    return FAIL_NONE;
}

static void pwm_init(void)
{
#if USE_HW_PWM
    PINSEL0 |= 2U << 16;
    PCONP |= 1U << 5;
    PWMTCR = 2; PWMPR = PCLK_HZ / 10000 - 1; PWMMR0 = 100; PWMMR4 = 0;
    PWMMCR = 2; PWMPCR = 1U << 12; PWMLER = 0x11; PWMTCR = 9;
#else
    IO0CLR = EN1_PIN;
#endif
    pwm_duty = 0;
}

static void motor_set_speed(unsigned int percent)
{
    if (percent > 100) percent = 100;
#if USE_HW_PWM
    PWMMR4 = percent >= 100 ? 101 : percent;
    PWMLER = 1U << 4;
#endif
    pwm_duty = percent;
    g_speed = percent;
    pwm_service();
}

static void motor_set_direction(unsigned int dir)
{
    if (dir == DIR_FORWARD) IO0CLR = IN2_PIN;
    else if (dir == DIR_REVERSE) IO0CLR = IN1_PIN;
    else IO0CLR = IN1_PIN | IN2_PIN;
    if (dir == DIR_FORWARD) IO0SET = IN1_PIN;
    if (dir == DIR_REVERSE) IO0SET = IN2_PIN;
    g_direction = dir;
}

static void motor_ramp(unsigned int target, unsigned int ramp_ms)
{
    unsigned int now = g_speed, step_ms;

    if (now == target) return;
    step_ms = udiv(ramp_ms, target > now ? target - now : now - target);
    while (now != target) {
        if (target > now) now++; else now--;
        motor_set_speed(now);
```

```
        delay_ms(step_ms);
    }
}

static void motor_run_step(const motor_step_t *s)
{
    if (s->direction != g_direction && g_speed) {
        motor_ramp(0, s->ramp_ms);
        motor_set_direction(DIR_STOP);
        delay_ms(DIR_CHANGE_PAUSE_MS);
    }
    motor_set_direction(s->direction);
    if (s->direction != DIR_STOP) motor_ramp(s->speed, s->ramp_ms);
    else motor_set_speed(0);
    delay_ms(s->run_ms);
}

static void blink(void)
{
    IO0SET = LED_PIN; delay_ms(100);
    IO0CLR = LED_PIN; delay_ms(100);
}

int main(void)
{
    unsigned int i;

    PINSEL0 &= ~((3U << 2) | (0xFFU << 8) | (3U << 16) | (0xFU << 24));
    PINSEL0 |= 0x1500;
    IO0SET = CS_PIN;
    IO0CLR = LED_PIN | EN1_PIN | IN1_PIN | IN2_PIN;
    IO0DIR |= LED_PIN | CS_PIN | EN1_PIN | IN1_PIN | IN2_PIN;

    VPBDIV = 0;
    timers_init();
    pwm_init();
    motor_set_direction(DIR_STOP);
    for (i = 0; i < 3; i++) blink();

    PCONP |= 1U << 8;
    S0SPCCR = 8;
    S0SPCR = 0x20;

    g_fail_code = profile_load();
    g_step_count = step_count;
    if (g_fail_code) for (;;) blink();

    IO0SET = LED_PIN;
    for (;;) {
        for (i = 0; i < step_count; i++) {
            g_step = i;
            motor_run_step(&steps[i]);
        }
        g_cycles++;
    }
}
```

8.2 Motor Driver Test (motor_test.c)

Listing 8.2: motor_test.c *(50 lines)*

```
#define REG32(addr) (*(volatile unsigned int *)(addr))

#define PINSEL0  REG32(0xE002C000)
#define IO0SET   REG32(0xE0028004)
#define IO0DIR   REG32(0xE0028008)
#define IO0CLR   REG32(0xE002800C)

#define LED_PIN  (1U << 1)
#define CS_PIN   (1U << 7)
#define EN1_PIN  (1U << 8)
#define IN1_PIN  (1U << 12)
#define IN2_PIN  (1U << 13)

#define PINSEL0_MASK  ((3U << 2) | (3U << 14) | (3U << 16) | (0xFU << 24))

static void delay(unsigned int count)
{
    volatile unsigned int i;

    for (i = 0; i < count; i++) {
    }
}

int main(void)
{
    PINSEL0 &= ~PINSEL0_MASK;
```

```
27
28      IO0SET = CS_PIN;
29      IO0CLR = LED_PIN | EN1_PIN | IN1_PIN | IN2_PIN;
30      IO0DIR |= LED_PIN | CS_PIN | EN1_PIN | IN1_PIN | IN2_PIN;
31
32      IO0SET = EN1_PIN;
33
34      while (1) {
35          IO0CLR = IN2_PIN;
36          IO0SET = IN1_PIN;
37          IO0SET = LED_PIN;
38          delay(2000000U);
39
40          IO0CLR = IN1_PIN | IN2_PIN;
41          IO0CLR = LED_PIN;
42          delay(1000000U);
43
44          IO0SET = IN2_PIN;
45          delay(2000000U);
46
47          IO0CLR = IN1_PIN | IN2_PIN;
48          delay(1000000U);
49      }
50  }
```

9. RESULTS AND DISCUSSION

9.1 EEPROM Communication

An EEPROM self-test program wrote 0x5A and then 0xA5 to address 0x0010 and read each value back. In Proteus the LED blinked three times and then stayed on, which the program allows only if the EEPROM answered, the write enable worked, both writes completed and both values were read back correctly. This confirmed the SPI0 wiring, chip-select handling and EEPROM supply.

9.2 Motor-Profile Execution

With the final firmware, the simulation reached `00:03:36` of simulated time at a CPU load of 29 %, so Proteus was keeping pace with real time (Figure 9.2). In Figure 9.1 the motor is rotating with a non-zero speed reading of about −48.8, IN1 is high and IN2 low (the forward step), and EN1 is low at that instant, as expected during the off part of each PWM period. Because IN1 is only driven after a successful profile load, this also shows that the EEPROM profile was read and accepted.

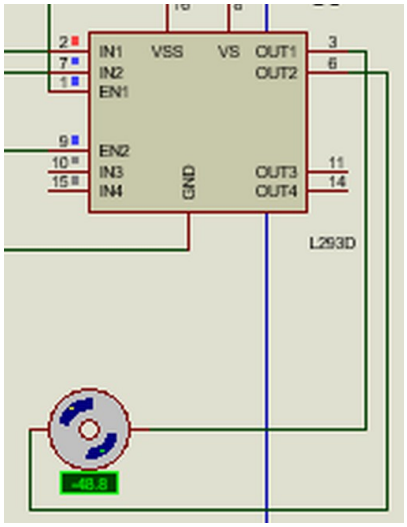


Figure 9.1: *L293D pin states and the rotating motor (detail of Figure 4.2)*



Figure 9.2: *Proteus status bar during the same run*

Before Proteus, the firmware was also built without warnings and run in an ARM emulator. There it handled blank, corrupted and unanswering EEPROMs correctly and ran the full profile; Figure 9.3 shows the measured duty cycle on EN1 over one cycle, which matches the profile of Table 5.2 and the 9.9 s cycle time calculated in Chapter 6. Not yet captured in Proteus are the reverse step, the LED state during the motor run and the Watch Window values.

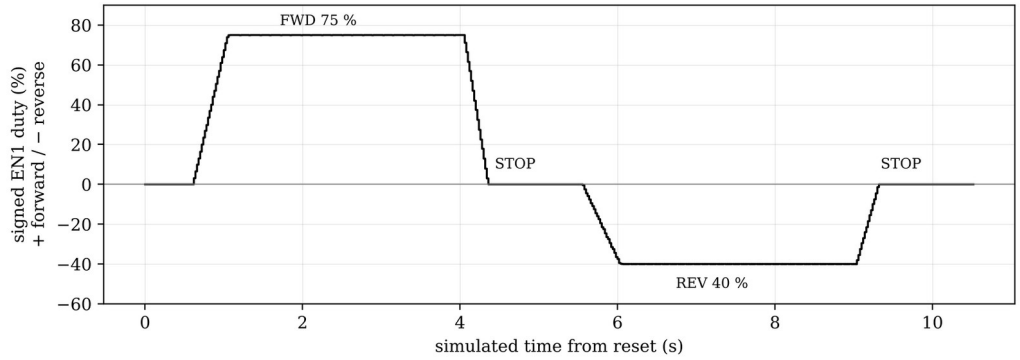


Figure 9.3: *Motor drive over one profile cycle, measured in emulation*

[Insert Proteus screenshot of the reverse step, with the Watch Window showing g_step, g_direction and g_speed]

Figure 9.4: *Reverse step of the profile in Proteus (to be captured)*

9.3 Problems Faced and Solutions

Table 9.1: *Main problems and how they were solved*

Problem	Solution
Proteus supply errors: VBAT at 0 V, VDD at 5 V, then a singular-matrix error on BAT1	All LPC2138 supply pins, including VBAT, connected to 3.3 V; supplies rearranged
First EEPROM test unclear: LED off, or on after a delay	Rewritten test with time-outs, two test values (0x5A, 0xA5) and a 3-blink start signal
`PINSEL0` = 0x5500 made P0.7 the SPI SSEL0 pin	`PINSEL0` = 0x1500, so P0.7 is a GPIO chip select
Build error `linker.ld:1: syntax error`	Wrong text had been pasted into the file; linker script re-created
Motor stayed at 0 RPM; Proteus far slower than real time with 3 kHz PWM	100 Hz software PWM; CPU load fell to 29 % and the motor ran

Figure 9.5 shows the last problem as it appeared in Proteus with the earlier firmware: IN1 is already high, so the profile had loaded, but the motor still reads 0.00 because the simulation had barely passed one second of simulated time.

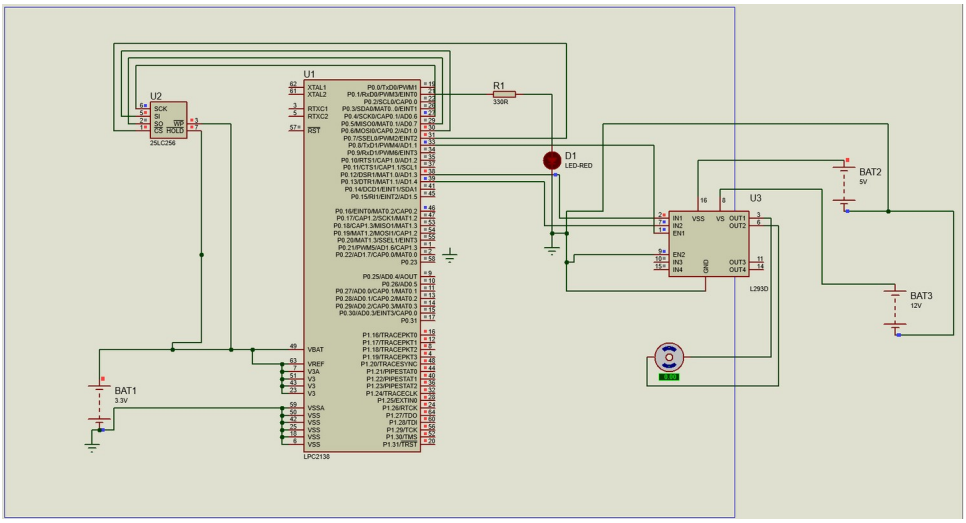


Figure 9.5: *Earlier run with 3 kHz PWM: IN1 high, motor still at 0.00*

9.4 Discussion

Two lessons stood out. First, a test must be able to fail visibly: the original EEPROM test could hang or pass on old data with no way to tell, while time-outs, two test values and a start-up blink made the result conclusive. Second, the motor at first appeared not to work although the firmware was already running the profile; the pin indicators and the simulator status bar showed that the real limit was simulation speed, which the lower PWM frequency solved.

10. APPLICATIONS AND FUTURE IMPROVEMENTS

10.1 Applications

- Conveyors, mixers and pumps that repeat a fixed speed and direction cycle.
- Fan or blower speed profiles that must survive power loss.
- Test rigs that run a motor through a stored sequence.
- Teaching register-level SPI, EEPROM handling and PWM motor control.

10.2 Limitations and Future Improvements

- Only simulation results are available; the circuit should be built and tested on real hardware.
- Speed control is open-loop; an encoder and a PI controller would regulate speed under load.
- Profiles can only be changed in code; a UART or push-button interface could edit several stored profiles.
- Timing assumes a 12 MHz clock without the PLL; a timer interrupt or hardware PWM would make PWM independent of the main loop.

11. CONCLUSION

An SPI EEPROM motor-profile controller based on the LPC2138 was designed, programmed in bare-metal C and verified in Proteus. SPI0 was configured at register level, and a complete 25LC256 driver was written with write-enable checks, write-cycle polling, page handling and time-outs. Motor profiles are stored in a versioned, checksummed format, loaded at power-up and executed on a DC motor through an L293D with 100 Hz PWM, speed ramps and safe direction changes. EEPROM communication and motor operation were both confirmed in simulation. The next step is to capture the full profile sequence in Proteus and build the circuit on real hardware.

REFERENCES

[1] NXP Semiconductors, *UM10120: LPC2131/2/4/6/8 User Manual*, Rev. 4, 23 April 2012.

[2] NXP Semiconductors, *LPC2131/2132/2134/2136/2138 single-chip 16/32-bit microcontrollers*, product data sheet.

[3] NXP Semiconductors, *AN10421: Power management for LPC2138*, Rev. 01, 6 January 2006.

[4] Microchip Technology Inc., *25AA256/25LC256 256K SPI Bus Serial EEPROM*, data sheet DS20001822J.

[5] Texas Instruments, *L293x Quadruple Half-H Drivers*, data sheet (L293D pin functions and supply ranges).

[6] Arm Ltd., *Arm GNU Toolchain* documentation (GCC, GNU assembler, GNU linker).

[7] Labcenter Electronics, *Proteus Design Suite* documentation, including virtual instruments.